



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

THUẬT TOÁN ỨNG DỤNG

THUẬT TOÁN QUY HOẠCH ĐỘNG

ONE LOVE. ONE FUTURE.

- Giới thiệu thuật toán quy hoạch động
- Tính số Fibonacci
- Đổi tiền
- Dãy con tăng dài nhất
- Dãy con chung dài nhất
- Quy hoạch động Bitmask

Lịch sử thuật toán quy hoạch động

- R.E.Bellman (1920-1984)



RICHARD BELLMAN ON THE BIRTH OF DYNAMIC PROGRAMMING

STUART DREYFUS

University of California, Berkeley, IEOR, Berkeley, California 94720, dreyfus@ieor.berkeley.edu

What follows concerns events from the summer of 1949, when Richard Bellman first became interested in multistage decision problems, until 1955. Although Bellman died on March 19, 1984, the story will be told in his own words since he left behind an entertaining and informative autobiography, *Eye of the Hurricane* (World Scientific Publishing Company, Singapore, 1984), whose publisher has generously approved extensive excerpting.

During the summer of 1949 Bellman, a tenured associate professor of mathematics at Stanford University with a developing interest in analytic number theory, was consulting for the second summer at the RAND Corporation in Santa Monica. He had received his Ph.D. from Princeton in 1946 at the age of 25, despite various war-related activities during World War II—including being assigned by the Army to the Manhattan Project in Los Alamos. He had already exhibited outstanding ability both in pure mathematics and in solving applied problems arising from the physical world. Assured of a successful conventional academic career, Bellman, during the period under consideration, cast his lot instead with the kind of applied mathematics later to be known as operations research. In those days applied practitioners were regarded as distinctly second-class citizens of the mathematical fraternity. Always one to enjoy controversy, when invited to speak at various university mathematics department seminars, Bellman delighted in justifying his choice of applied over pure mathematics as being motivated by the real world's greater challenges and mathematical demands.

what RAND was interested in. He suggested that I work on multistage decision processes. I started following that suggestion" (p. 157).

CHOICE OF THE NAME DYNAMIC PROGRAMMING

"I spent the Fall quarter (of 1950) at RAND. My first task was to find a name for multistage decision processes.

"An interesting question is, 'Where did the name, dynamic programming, come from?' The 1950s were not good years for mathematical research. We had a very interesting gentleman in Washington named Wilson. He was Secretary of Defense, and he actually had a pathological fear and hatred of the word, research. I'm not using the term lightly; I'm using it precisely. His face would suffuse, he would turn red, and he would get violent if people used the term, research, in his presence. You can imagine how he felt, then, about the term, mathematical. The RAND Corporation was employed by the Air Force, and the Air Force had Wilson as its boss, essentially. Hence, I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics inside the RAND Corporation. What title, what name, could I choose? In the first place I was interested in planning, in decision making, in thinking. But planning, is not a good word for various reasons. I decided therefore to use the word, 'programming.' I wanted to get across the idea that this was dynamic, this was multistage, this was time-varying—I thought, let's kill two birds with one stone. Let's take a word that has an

Lịch sử thuật toán quy hoạch động

- Trong chiến tranh thế giới thứ 2, các ngành khoa học cơ bản ở Mỹ không được đầu tư để dành toàn bộ nguồn lực cho thể chiến, chỉ những kết quả khoa học ứng dụng trực tiếp cho chiến trường mới được cấp kinh phí nghiên cứu, ví dụ: qui hoạch tuyến tính với bài toán khẩu phần ăn cho binh sĩ.
- Nhà toán học Bellman thời kỳ đó nghiên cứu ra phương pháp ‘multistage decision processes’ (quá trình ra quyết định thông qua nhiều lớp) trong lĩnh vực lập kế hoạch (planning). Tuy nhiên từ ‘planning’ không phù hợp vào thời kỳ đó nên ông đã thay bằng từ ‘programming’ (lập trình) thời thượng hơn khi mà máy tính to đầu tiên của quân đội Mỹ ra đời. Tiếp theo ông thay từ ‘multistage’ bằng từ ‘dynamic’ nghe hay hơn thể hiện sự gổn nhau về thời gian. Thuật ngữ ‘dynamic programming’ ra đời từ đó.
- Dynamic programming mang tính kỹ thuật lập trình nhiều hơn là tính mô hình dạng bài toán (như qui hoạch tuyến tính), tuy nhiên từ dịch ra ‘Qui Hoạch Động’ nghe hay và thuận hơn từ ‘Lập Trình Động’.

Quy hoạch động là gì?

- Là một mô hình giải bài
- Nhiều điểm tương đồng với hai phương pháp Chia để trị và Quay lui
- Nhắc lại Chia để trị:
 - Chia bài toán cha thành các bài toán con **độc lập**
 - Giải từng bài toán con (bằng đệ qui)
 - Kết hợp lời giải các bài toán con lại thành lời giải của bài toán cha
- Phương pháp quy hoạch động:
 - Chia bài toán cha thành các bài toán con **gối nhau**
 - Giải từng bài toán con (bằng đệ qui)
 - Kết hợp lời giải các bài toán con lại thành lời giải của bài toán cha
 - **Không tìm nhiều hơn một lần lời giải của cùng một bài toán**

Công thức Quy hoạch động

- Tìm công thức qui hoạch động cho bài toán dựa trên các bài toán con

1. Cài đặt công thức qui hoạch động:
2. Đơn giản là chuyển công thức thành hàm đệ qui
3. Lưu trữ kết quả các hàm đã tính toán

- **Nhận xét:**

Bước 1: tìm công thức qui hoạch động là bước khó nhất và quan trọng nhất.

Bước 2 và 3 có thể áp dụng sơ đồ chung sau đây để thực hiện.

Cài đặt Top-Down với Hệ qui có nhớ

```
map <problem, value> Memory;
value DP(problem P) {
    if (is_base_case(P))
        return base_case_value(P);

    if (Memory.find(P) != Memory.end())
        return Memory [P];

    value result = some value ;
    for (problem Q in subproblems(P))
        result = Combine(result, DP(Q));

    Memory [P] = result ;
    return result ;
}
```

- Việc sử dụng hàm đệ qui để cài đặt công thức qui hoạch động là cách tiếp cận lập trình tự nhiên và đơn giản cho lập trình giải bài toán qui hoạch động, ta gọi đó là cách tiếp cận lập trình Top-Down, phù hợp với đa số người mới tiếp cận kỹ thuật Qui hoạch động
- Khi đã quen thuộc với các bài qui hoạch động ta có thể luyện tập phương pháp lập trình Bottom-Up, xây dựng dần lời giải từ các bài toán con đến các bài toán cha
- Các bước trên mới chỉ tìm ra được giá trị tối ưu của bài toán. Nếu phải đưa ra các phần tử trong lời giải tạo nên giá trị tối ưu của bài toán thì cần thực hiện thêm bước Truy vết. Bước Truy vết nên mô phỏng lại Bước 2 cài đặt đệ qui và tìm ra các phần tử của lời giải dựa trên thông tin các bài toán con đã được lưu trữ trong mảng memory

- Giới thiệu thuật toán quy hoạch động
- **Tính số Fibonacci**
- Đổi tiền
- Dãy con tăng dài nhất
- Dãy con chung dài nhất
- Quy hoạch động Bitmask

Tính số Fibonacci

- *Hai số đầu tiên của dãy Fibonacci là 1 và 1. Tất cả các số khác của dãy được tính bằng tổng của hai số ngay trước nó trong dãy*
 - **Yêu cầu:** Tính số Fibonacci thứ n
 - Thử giải bài toán bằng phương pháp Quy hoạch động
1. Tìm công thức truy hồi:

$$\text{Fib}(1) = 1$$

$$\text{Fib}(2) = 1$$

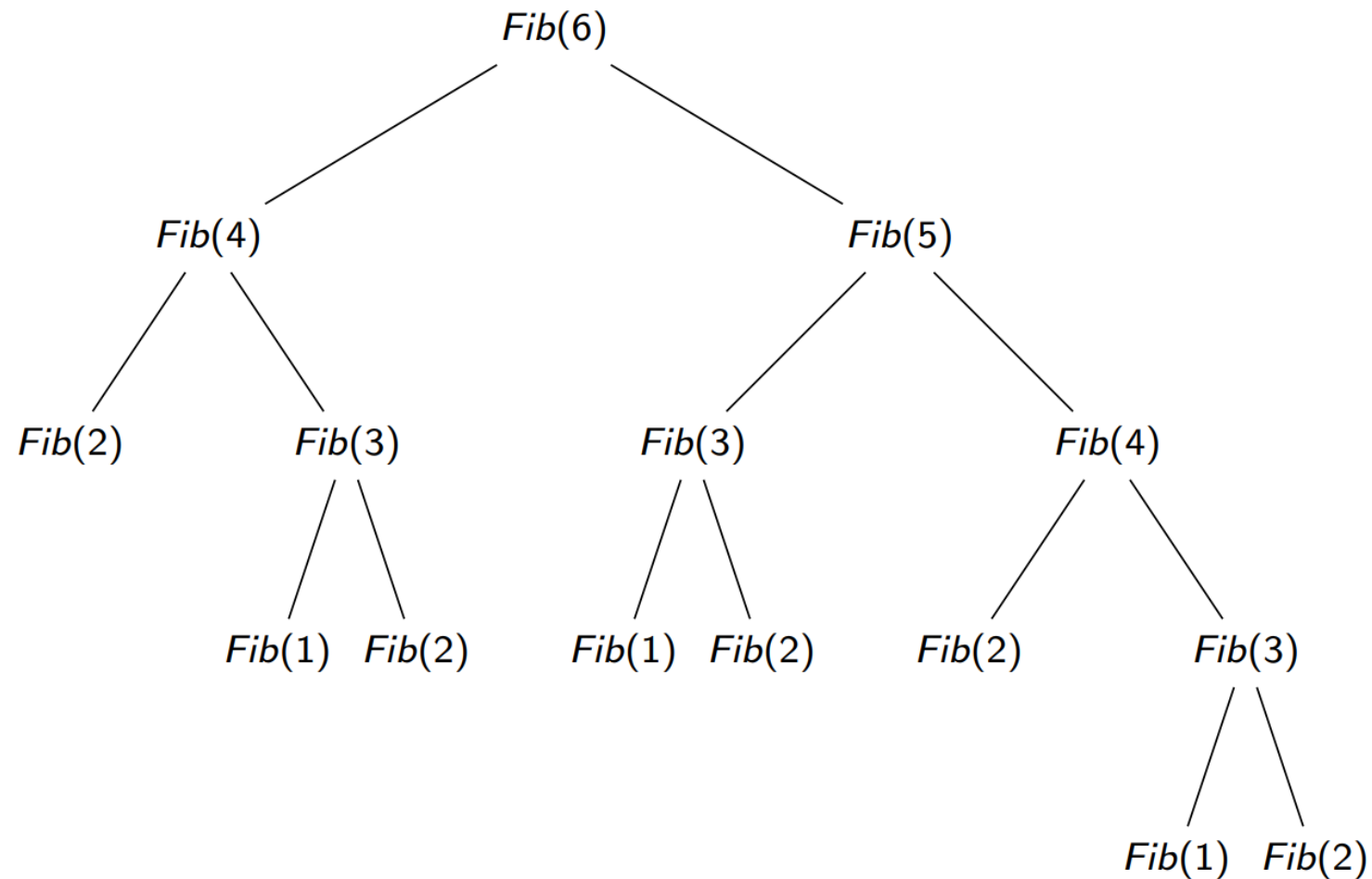
$$\text{Fib}(n) = \text{Fib}(n - 2) + \text{Fib}(n - 1)$$

2. Cài đặt công thức qui hoạch động

```
int Fib(n) {  
    if (n <= 2) return 1;  
    return Fib(n - 1) + Fib(n - 2);  
}
```

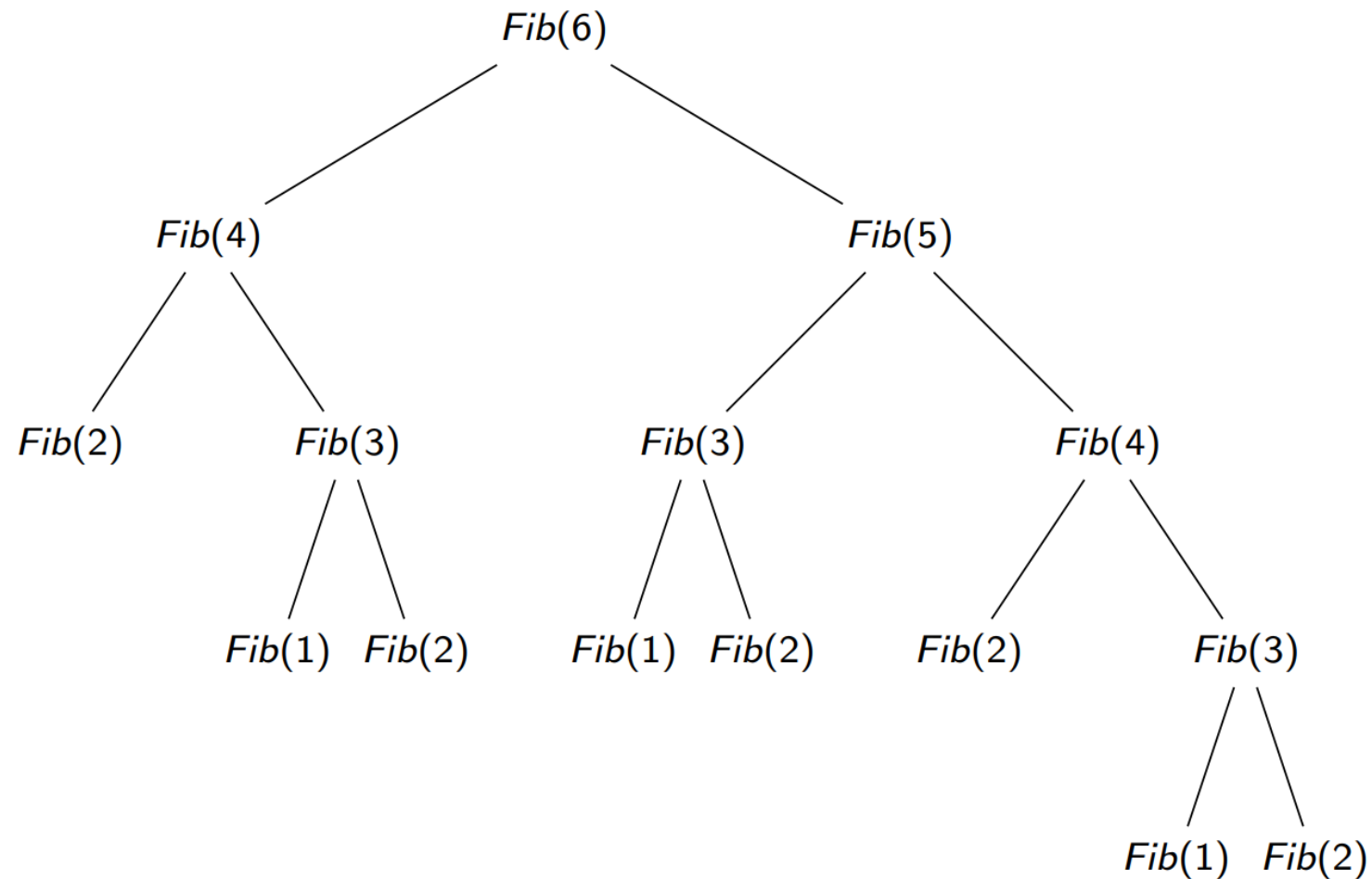
Tính số Fibonacci

Độ phức tạp là bao nhiêu?



Tính số Fibonacci

Độ phức tạp là bao nhiêu? Hàm mũ, gần như $O(2^n)$



Tính số Fibonacci

- Lưu trữ các kết quả đã tính

```
map<int, int> mem;
```

```
int Fib(int n){  
    if (n <= 2) return 1;  
  
    if (mem.find(n) != mem.end())  
        return mem[n];  
  
    int res = Fib(n - 1) + Fib(n - 2);  
    mem[n] = res;  
  
    return res;  
}
```



Tính số Fibonacci

```
const int N = 1e4 + 5;
int mem[N];
memset(mem, -1, sizeof(mem));

int Fib(int n){
    if (n <= 2) return 1;

    if (mem[n] == -1) return mem[n];

    int res = Fib(n - 1) + Fib(n - 2);
    mem[n] = res;

    return res;
}
```

- Bây giờ độ phức tạp bao nhiêu?

Tính số Fibonacci: Độ phức tạp

- Có n khả năng đầu vào input cho hàm đệ qui: $1, 2, \dots, n$
- Với mỗi input:
- hoặc là kết quả được tính và lưu trữ lại
- hoặc là lấy luôn ra từ bộ nhớ nếu như trước đây đã được tính
- Mỗi input sẽ được tính tốt đa một lần
- Thời gian tính là $O(n \times f)$, với f là thời gian tính toán của hàm với một input, với giả thiết là kết quả đã tính trước đây sẽ được lấy trực tiếp từ bộ nhớ, chỉ trong $O(1)$
- Do ta chỉ tốn một lượng hằng số phép tính đối với một input của hàm, nên $f = O(1)$
- Thời gian tính tổng cộng là $O(n)$

Đoạn con có tổng lớn nhất

- Cho một dãy số nguyên $A[1], A[2], \dots, A[n]$, hãy tìm một đoạn trong dãy có trọng số lớn nhất, nghĩa là tổng các số trong đoạn là lớn nhất

-16	7	-3	0	-1	5	-4
-----	---	----	---	----	---	----

- Tổng của đoạn có trọng số lớn nhất trong dãy là 8
- Nhớ lại:
 - Phương pháp Chia để trị cho độ phức tạp $O(n \log n)$
 - Liệu có thể làm tốt hơn với phương pháp Quy hoạch động?

Đoạn con có tổng lớn nhất: Công thức sai

- Bước đầu tiên là đi tìm công thức Qui hoạch động
- Gọi $MaxSum(i)$ là trọng số của đoạn có trọng số lớn nhất giới hạn trong đoạn $1, \dots, i$.
- Bước cơ sở: $MaxSum(1) = \max(0, A[1])$
- Bước chuyển qui nạp: $MaxSum(i)$ có liên hệ gì với $MaxSum(i - 1)$
- Liệu có thể kết hợp lời giải của các bài toán con có kích thước bé hơn i thành lời giải bài toán có kích thước bằng i
- Câu trả lời không hoàn toàn hiển nhiên ...

Đoạn con có tổng lớn nhất: Công thức

- Hãy thay đổi hàm mục tiêu:
- Gọi $MaxSum(i)$ là trọng số của đoạn có trọng số lớn nhất giới hạn trong đoạn $1, \dots, i$ mà kết thúc tại $A[i]$.
- Bước cơ sở: $MaxSum(1) = A[1]$
- Bước chuyển qui nạp: $MaxSum(i) = \max(A[i], A[i] + MaxSum(i - 1))$
- Vậy kết quả cuối cùng chính là: $\max_{1 \leq i \leq n} \{MaxSum(i)\}$

Đoạn con có tổng lớn nhất: Cài đặt

- Bước tiếp theo là cài đặt công thức Qui hoạch động

```
const int N = 1e4 + 5;
```

```
int a[N];
```

```
int MaxSum(int i){
```

```
    if (i == 1) return a[1];
```

```
    int res = max(a[i], a[i] + MaxSum(i - 1));
```

```
    return res;
```

```
}
```

- Không cần sử dụng mảng nhớ memory , vì sao?
- Nhưng vẫn cần mảng nhớ memory phục vụ cho bước truy vết!

Đoạn con có tổng lớn nhất: Cài đặt

```
const int N = 1e6 + 5;
int a[N];
int mem[N];
bool mark[N];
memset(mark, 0, sizeof(mark));

int MaxSum(int i){
    if (i == 1) return a[1];

    if (mark[i]) return mem[i];

    int res = max(a[i], a[i] + MaxSum(i - 1));
    mark[i] = true;
    mem[i] = res;

    return res;
}
```

Đoạn con có tổng lớn nhất: Kết quả

- Thủ tục chính chỉ cần gọi đệ qui một lần cho $MaxSum(n)$, hàm đệ qui sẽ tính toàn bộ các giá trị của $MaxSum(i)$, $1 \leq i \leq n$.
- Kết quả bài toán là giá trị lớn nhất trong các giá trị $MaxSum(i)$ đã được lưu trữ trong $mem[i]$ sau quá trình gọi đệ qui.

```
int ans = 0;
```

```
for (int i = 1; i <= n; i++) {  
    ans = max(ans, mem[i]);  
}
```

```
cout << ans;
```

- Nếu bài toán yêu cầu tìm đoạn có trọng số lớn nhất trong nhiều dãy khác nhau, thì hãy nhớ xóa bộ nhớ khi kết thúc tính toán ở mỗi mảng.

Đoạn con có tổng lớn nhất: Độ phức tạp

- Có n khả năng đầu vào input cho hàm đệ qui
- Mỗi input được tính trong $O(1)$
- Thời gian tính toán tổng cộng là $O(n)$
- Làm thế nào để biết chính xác một đoạn nào trong dãy tạo ra giá trị tổng lớn nhất tìm được?

Đoạn con có tổng lớn nhất: Truy vết bằng đệ quy

- Làm giống như hàm đệ quy chính và sử dụng mảng *mem[i]* đã có để truy vết ngược lại

```
void Trace(int i) {  
    if (i != 1 && mem[i] == a[i] + mem[i - 1]){  
        Trace(i - 1);  
    }  
    cout << a[i] << ' ' ;  
}
```

- Độ phức tạp hàm truy vết? $O(n)$

Đoạn con có tổng lớn nhất: Truy vết bằng vòng lặp

```
int ans = 0, pos = -1;

for (int i = 1; i <= n; i++) {
    if (ans < mem[i]) {
        ans = mem[i];
        pos = i;
    }
}

cout << ans << endl;

int left = pos, right = pos, sum = a[left];
while (sum != ans){
    left--;
    sum += a[left];
}

cout << left << ' ' << right << endl;
```

- Giới thiệu thuật toán quy hoạch động
- Tính số Fibonacci
- **Đổi tiền**
- Dãy con tăng dài nhất
- Dãy con chung dài nhất
- Quy hoạch động Bitmask

- Cho trước một tập các đồng tiền mệnh giá $D_1, D_2, \dots, D_n$ và một mệnh giá x . Hãy tìm số lượng ít nhất các đồng tiền để đổi cho mệnh giá x .
- Giống bài toán cái túi?
- Tồn tại thuật toán tham lam cho bài Đổi tiền này?
- Bài toán cái túi đã học trong môn Toán rời rạc được giải bằng thuật toán duyệt nhánh cận. Còn thuật toán tham lam không hề chắc chắn đưa ra lời giải tối ưu, thậm chí nhiều trường hợp còn không đưa ra được lời giải...
- Hãy thử sử dụng phương pháp Quy hoạch động !
- Cuối cùng đưa ra nhận xét giữa các phương pháp khác nhau tiếp cận giải bài toán này

Đổi tiền: công thức quy hoạch động

- Bước đầu tiên: xây dựng công thức Qui hoạch động
- Gọi $MinCoin(i, x)$ là số lượng tiền ít nhất cần để đổi mệnh giá x nếu chỉ được phép sử dụng các đồng tiền mệnh giá $D_1, D_2, \dots, D_i$
- Các bước cơ sở:

$$MinCoin(i, x) = \infty \text{ nếu } x < 0$$

$$MinCoin(i, 0) = 0$$

$$MinCoin(0, x) = \infty$$

- Bước chuyển quy nạp:

$$MinCoin(i, x) = \min \begin{cases} 1 + MinCoin(i, x - D_i) \\ MinCoin(i - 1, x) \end{cases}$$

Đổi tiền: Cài đặt

```
const int INF = 1e9;
const int N = 20;
int D[N];

int MinCoin(int i, int x){
    if (x < 0 || i == 0) return INF;
    if (x == 0) return 0;

    int res = INF;
    res = min(res, 1 + MinCoin(i, x - D[i]));
    res = min(res, MinCoin(i - 1, x));

    return res;
}
```

Đổi tiền: Cài đặt

```
const int INF = 1e9; const int N = 20;
const int XMAX = 1e5 + 5;
int D[N], mem[N][XMAX];
memset(mem, -1, sizeof(mem));

int MinCoin(int i, int x){
    if (x < 0 || i == 0) return INF;
    if (x == 0) return 0;

    if (mem[i][x] == -1) return mem[i][x];
    int res = INF;
    res = min(res, 1 + MinCoin(i, x - D[i]));
    res = min(res, MinCoin(i - 1, x));

    return mem[i][x] = res;
}
```

Đổi tiền: Độ phức tạp

- Độ phức tạp?
- Số lượng khả năng đầu vào input là $n \times x$
- Mỗi input được xử lý trong $O(1)$, giả thiết mỗi lời gọi đệ qui thực hiện trong thời gian hằng số
- Thời gian tính toán tổng cộng là $O(n \times x)$
- Làm thế nào để xác định được những đồng tiền nào cho phương án tối ưu?
- Hãy truy vết ngược lại quá trình đệ qui

Đổi tiền: Truy vết bằng đệ quy

```
void Trace(int i, int x) {  
    if (x <= 0 || i == 0) return;  
  
    if (mem[i][x] == 1 + mem[i][x - D[i]]){  
        cout << D[i] << ' ';  
        Trace(i, x - D[i]);  
    } else Trace(i - 1, x);  
}
```

- Gọi Trace(n, x);
- Độ phức tạp hàm truy vết? $O(\max(n, x))$

Dãy con tăng dài nhất

- Cho một dãy n số nguyên $A[1], A[2], \dots, A[n]$. Hãy tìm độ dài dãy con tăng dài nhất.
- Định nghĩa: Nếu xoá đi 0 phần tử hoặc một số phần tử của dãy A thì sẽ thu được một dãy con của A .
- Ví dụ: $A = [2, 0, 6, 1, 2, 9]$
- $[2, 6, 9]$ là một dãy con
- $[2, 2]$ là một dãy con
- $[2, 0, 6, 1, 2, 9]$ là một dãy con
- $[]$ là một dãy con
- $[9, 0]$ không là một dãy con
- $[7]$ không là một dãy con

Dãy con tăng dài nhất

- Một dãy con tăng của A là một dãy con của A sao cho các phần tử là tăng chặt từ trái sang phải
- [2, 6, 9] và [1, 2, 9] là hai dãy con tăng của A = [2, 0, 6, 1, 2, 9]
- Làm thế nào để tính độ dài dãy con tăng dài nhất?
- Có 2^n dãy con, phương pháp đơn giản nhất là duyệt qua toàn bộ các dãy này
- Thuật toán cho độ phức tạp $O(n \times 2^n)$, chỉ có thể chạy nhanh được ra kết quả với $n \leq 23$.
- Hãy thử phương pháp Quy hoạch động !

Dãy con tăng dài nhất: Công thức Quy hoạch động

- Gọi $LIS(i)$ là độ dài dãy con tăng dài nhất của mảng $A[1], A[2], \dots, A[i]$.
- Bước cơ sở: $LIS(1) = A[1]$
- Bước chuyển qui nạp cho $LIS(i)$?
- Nếu đặt hàm mục tiêu như vậy sẽ gặp phải vấn đề giống như bài toán dãy con có tổng lớn nhất ở trên, hãy thay đổi một chút hàm mục tiêu.

Dãy con tăng dài nhất: Công thức Quy hoạch động

- Gọi $LIS(i)$ là độ dài dãy con tăng dài nhất của mảng $A[1], A[2], \dots, A[i]$ mà kết thúc tại phần tử thứ i .
- Bước cơ sở: Không cần thiết
- Bước chuyển qui nạp:

$$LIS(i) = \max(1, \max_{j \text{ s.t. } A[j] < A[i]} \{1 + LIS(j)\})$$

Dãy con tăng dài nhất: Cài đặt

```
const int N = 1e4 + 5;
int a[N], mem[N];
memset(mem, -1, sizeof(mem));

int LIS(i) {
    if (mem[i] == -1) return mem[i];

    int res = 1;
    for (int j = 1; j < i; j++){
        if (a[j] < a[i])
            res = max(res, 1 + LIS(j));
    }
    mem[i] = res;

    return res;
}
```

Dãy con tăng dài nhất: Cài đặt

- Độ dài dãy con tăng dài nhất chính là giá trị lớn nhất trong các giá trị LIS(i)

```
int ans = 0, pos = 0;

for (int i = 1; i <= n; i++){
    if (ans < mem[i]){
        ans = mem[i];
        pos = i;
    }
}
cout << ans << endl;
```

Dãy con tăng dài nhất: Độ phức tạp

- Có n khả năng cho đầu vào input
- Mỗi input được tính trong thời gian $O(n)$
- Thời gian tính tổng cộng là $O(n^2)$
- Có thể chạy được đến $n \leq 10000$, tốt hơn rất nhiều so với phương duyệt toàn bộ
- Áp dụng cấu trúc cây phân đoạn vào phương pháp trên sẽ cải tiến độ độ phức tạp thành $O(n \log n)$
- Phương pháp cải tiến khác là đặt công thức Quy hoạch động mới kết hợp chặt nhị phân cũng cho độ phức tạp $O(n \log n)$
- Truy vết ?

Dãy con tăng dài nhất: Truy vết bằng đệ quy

```
void Trace(int i) {  
    for (int j = 1; j < i; j++){  
        if (a[j] < a[i] && mem[i] == 1 + mem[j]){  
            Trace(j);  
            break;  
        }  
    }  
    cout << a[i] << ' ' ;  
}
```

- Gọi Trace(pos);
- Độ phức tạp hàm truy vết? $O(n^2)$
- Có thể cải tiến thành $O(n)$

Dãy con tăng dài nhất: Truy vết bằng đệ quy cải tiến

```
void Trace(int i) {  
    for (int j = i - 1; j >= 1; j--){  
        if (a[j] < a[i] && mem[i] == 1 + mem[j]){  
            Trace(j);  
            break;  
        }  
    }  
    cout << a[i] << ' ' ;  
}
```

- Gọi Trace(pos);
- Độ phức tạp hàm truy vết? $O(n)$

Dãy con tăng dài nhất: Truy vết bằng vòng lặp

```
stack<int> stk;  
int i = pos;  
  
for (int k = 0; k < ans; k++) {  
    stk.push(i);  
    for(int j = i - 1; j >= 1; j--) {  
        if (a[j] < a[i] && mem[i] == 1 + mem[j]) {  
            i = j;  
            break;  
        }  
    }  
}  
  
while (!stk.empty()) {  
    cout << stk.top() << ' ';  
    stk.pop();  
}
```

Dãy con chung dài nhất

- Cho hai xâu (hoặc hai mảng số nguyên) n phần tử $X[1], X[2], \dots, X[n]$ và $Y[1], Y[2], \dots, Y[m]$. Hãy tìm độ dài dãy con chung dài nhất của hai xâu.
- Ví dụ: $X = \text{"abcb"}\text{"}$, $Y = \text{"bdcab"}\text{"}$
- Dãy con chung dài nhất của X và Y là $\text{"bcb"}\text{"}$ có độ dài là 3.

Dãy con chung dài nhất: Công thức Quy hoạch động

- Gọi $LCS(i, j)$ là độ dài dãy con chung dài nhất của dãy $X[1], X[2], \dots, X[i]$ và $Y[1], Y[2], \dots, Y[j]$.

- Bước cơ sở:

$$L(0, j) = L(i, 0) = 0$$

- Bước chuyển quy nạp:

$$LCS(i, j) = \max \begin{cases} LCS(i, j - 1) \\ LCS(i - 1, j) \\ 1 + LCS(i - 1, j - 1) \quad \text{nếu } X[i] = Y[j] \end{cases}$$

Dãy con chung dài nhất: Cài đặt

```
const int N = 1e4 + 5;
string X, Y;
int mem[N][N];
memset(mem, -1, sizeof(mem));

int LCS(int i, int j) {
    if (i == 0 || j == 0) return 0;
    if (mem[i][j] != -1) return mem[i][j];
    int res = 0;
    res = max(res, LCS(i - 1, j));
    res = max(res, LCS(i, j - 1));
    if (X[i-1] == Y[j-1]) {
        res = max(res, 1 + LCS(i - 1, j - 1));
    }
    return mem[i][j] = res;
}
```

Dãy con chung dài nhất: Ví dụ

iMem	j	0	1	2	3	4	5
i		Y[j]	<u>b</u>	d	<u>c</u>	a	<u>b</u>
0	X[i]	0	0	0	0	0	0
1	a	0 ↘	0	0	0	1	1
2	<u>b</u>	0	1 →	1 ↘	1	1	2
3	<u>c</u>	0	1	1	2 →	2 ↘	2
4	<u>b</u>	0	1	1	2	2	3

$$\text{LCS}(i, j) = \max \begin{cases} \text{LCS}(i, j - 1) \\ \text{LCS}(i - 1, j) \\ 1 + \text{LCS}(i - 1, j - 1) \quad \text{nếu } X[i] = Y[j] \end{cases}$$

Dãy con chung dài nhất: Độ phức tạp

- Có $n \times m$ khả năng cho đầu vào input
- Mỗi input được tính trong thời gian $O(1)$
- Thời gian tính tổng cộng là $O(n \times m)$
- Làm thế nào để biết chính xác những phần tử nào thuộc dãy con chung dài nhất?

Dãy con chung dài nhất: Truy vết bằng đệ quy

```
void Trace(int i, int j) {  
    if (i == 0 || j == 0) return;  
    if (X[i-1] == Y[j-1] && mem[i][j] == 1 + mem[i-1][j-1]) {  
        Trace(i - 1, j - 1);  
        cout << X[i-1];  
        return;  
    }  
    if (mem[i][j] == mem[i-1][j]) {  
        Trace(i - 1, j);  
        return;  
    }  
    if (mem[i][j] == mem[i][j-1]) {  
        Trace(i, j - 1);  
        return;  
    }  
}
```

Dãy con chung dài nhất: Truy vết bằng đệ quy

- Độ phức tạp hàm truy vết? $O(n + m)$

Dãy con chung dài nhất: Truy vết bằng vòng lặp

```
void IterTrace() {
    int i = X.size(), j = Y.size();
    stack<int> stk;
    int ans = LCS(i, j);
    for (int k = 0; k < ans; k++) {
        if (X[i - 1] == Y[j - 1] && mem[i][j] == 1 + mem[i-1][j-1]) {
            stk.push(X[i - 1]);
            i--; j--; continue;
        }
        if (mem[i][j] == mem[i-1][j]) {
            i--; continue;
        }
        if (mem[i][j] == mem[i][j-1]) {
            j--; continue;
        }
    }
    while (!stk.empty()){
        cout << stk.top() << ' ';
        stk.pop();
    }
}
```

Quy hoạch động trên bitmask

- Có còn nhớ biểu diễn bitmask cho các tập con?
- Mỗi tập con của tập n phần tử được biểu diễn bởi một số nguyên trong khoảng $0, 1, \dots, 2^n - 1$
- Điều này có thể giúp thực hiện phương pháp quy hoạch động dễ dàng trên các tập con



Applying the Traveling Salesman Problem to Business Analytics

Published on April 2, 2016

- *“The history and evolution of solutions to the Traveling Salesman Problem can provide us with some valuable concepts for business analytics and algorithm development”*
- *“Just as the traveling salesman makes his journey, new analytical requirements arise that require a journey into the development of solutions for them. As the data science and business analytics landscape evolves with new solutions, we can learn from the history of these journeys and apply the same concepts to our ongoing development”*



Applying the Traveling Salesman Problem to Business Analytics

Published on April 2, 2016

- Bài toán người đi du lịch là bài toán NP-khó kinh điển nhưng có rất nhiều ứng dụng trong thực tế, đặc biệt ngày nay với ứng dụng của khoa học dữ liệu và phân tích tài chính.
- Mỗi khi một hành trình của người bán hàng kết thúc, các dữ liệu sẽ được phân tích bởi các thuật toán trong khoa học dữ liệu, áp dụng vào ngành phân tích tài chính, từ đó có thể 'học máy' các kết quả lịch sử để áp dụng vào kế hoạch phát triển tiếp theo.

Bài toán người đi du lịch

- Cho một đồ thị n đỉnh $\{0, 1, \dots, n - 1\}$ và giá trị trọng số $C_{i,j}$ trên mỗi cặp đỉnh i, j . Hãy tìm một chu trình đi qua tất cả các đỉnh của đồ thị, mỗi đỉnh đúng một lần sao cho tổng các trọng số trên chu trình đó là nhỏ nhất
- Đây là bài toán NP-khó, vì vậy không tồn tại thuật toán tất định thời gian đa thức nào hiện biết để giải bài toán này
- Thuật toán duyệt toàn bộ đơn giản duyệt qua toàn bộ các hoán vị các đỉnh cho độ phức tạp là $O(n!)$, nhưng chỉ có thể chạy được đến $n \leq 11$.
- Liệu có thể làm tốt hơn với phương pháp Quy hoạch động?

Bài toán người đi du lịch: Công thức Quy hoạch động

- Không mất tính tổng quát giả sử chu trình bắt đầu và kết thúc tại đỉnh 0.
- Gọi $TSP(i, S)$ là cách sử dụng ít chi phí nhất để đi qua toàn bộ các đỉnh và quay trở lại đỉnh 0, nếu như hiện tại hành trình đang ở tại đỉnh i và người du lịch đã thăm tất cả các đỉnh trong tập S .
- Bước cơ sở: $TSP(i, \text{tập mọi đỉnh}) = C_{i,0}$
- Bước chuyển quy nạp:

$$TSP(i, S) = \min_{j \notin S} \{ C_{i,j} + tsp(j, S \cup \{j\}) \}$$

Bài toán người đi du lịch: Cài đặt

```
const int N = 20;
const int INF = 1e9;
int n;
int C[N][N], mem[N][1 << N];

int TSP(int i, int S) {
    if (S == (1 << n) - 1)
        return mem[i][S] = C[i][0];

    if (mem[i][S] != -1) return mem[i][S];
    int res = INF;
    for (int j = 1; j <= n; j++) {
        if (S & (1 << j)) continue;
        res = min(res, C[i][j] + TSP(j, S | (1 << j)));
    }
    return mem[i][S] = res;
}
```

Bài toán người đi du lịch: Cài đặt

- Kết quả tối ưu có thể đưa ra như sau:
`cout << TSP(0, 1) << endl;`

Bài toán người đi du lịch: Độ phức tạp tính toán

- Có $n \times 2^n$ khả năng cho đầu vào input
- Mỗi input được tính trong thời gian $O(n)$
- Thời gian tính tổng cộng là $O(n^2 \times 2^n)$
- Như vậy có thể tính nhanh được với n lên đến 20
- Làm thế nào để đưa ra được chính xác hành trình của người du lịch?

Bài toán người đi du lịch: Truy vết bằng đệ quy

```
void Trace(int i, int S) {
    cout << i << " -> ";
    if (S == (1 << n) - 1)
        cout << 0 << endl;
    for (int j = 1; j <= n; j++) {
        if (S & (1 << j)) continue;
        if (mem[i][S] == C[i][j] + mem[j][S | (1 << j)]) {
            trace(j, S | (1 << j));
            break;
        }
    }
}
```

Bài toán người đi du lịch: Truy vết bằng đệ quy

```
void Trace(int i, int S) {
    cout << i << " -> ";
    if (S == (1 << n) - 1)
        cout << 0 << endl;
    for (int j = 1; j <= n; j++) {
        if (S & (1 << j)) continue;
        if (mem[i][S] == C[i][j] + mem[j][S | (1 << j)]) {
            trace(j, S | (1 << j));
            break;
        }
    }
}
```

- Gọi Trace(0, 1 << 0);
- Độ phức tạp hàm truy vết? $O(n^2)$

Bài toán người đi du lịch: Truy vết bằng vòng lặp

```
void IterTrace() {  
    int i = 0, S = 1;  
    cout << 0 << " -> ";  
    for (int k = 1; k < n; k++) {  
        for (int j = 1; j < n; j++) {  
            if (S & (1 << j)) continue;  
            if (mem[i][S] == C[i][j] + mem[j][S | (1 << j)]) {  
                i = j;  
                S |= (1 << j);  
                break;  
            }  
        }  
    }  
    cout << i << " -> ";  
}  
cout << 0 << endl;
```

A graphic on the left side of the slide. It features a dark blue background with a large, stylized circular shape composed of many small red dots. The dots are arranged in a way that creates a sense of depth and movement, resembling a spiral or a stylized 'H' shape. The word 'HUST' is written in white, bold, sans-serif capital letters in the center of this graphic.

HUST

THANK YOU !